РО 1.3. Базовое администрирование Linux

Лекция. Права доступа Linux: chmod, chown, специальные биты, совместные каталоги

Цель лекции

Научиться объяснять модель прав Linux и настраивать владельца, группу, режим доступа и специальные биты для учебного сценария.

Список учебных вопросов

1. Модель прав доступа Linux: владелец, группа, остальные
2. Права r, w, x для файлов
3. Права r, w, x для каталогов
4. Символьная запись прав
5. Числовая запись прав
6. Изменение прав командой chmod
7. Изменение владельца и группы командой chown
8. Специальные биты: SUID, SGID, sticky bit
9. Совместные каталоги для группы и типовые ошибки настройки прав

1. Модель прав доступа Linux

Каждый файл и каталог имеет владельца, группу и права для трех категорий.

Категории: user или владелец, group или группа, other или остальные.

Права определяют, кто может читать, изменять или выполнять объект.

Для администратора это базовый механизм защиты файлов и каталогов.

Как читать ls -l

Пример: **-rw-r----- 1 student developer 1200 file.txt**

Первый символ показывает тип объекта.

Далее идут три тройки прав: владелец, группа, остальные.

student — владелец, developer — группа.

По этой строке можно понять, кто имеет доступ к объекту.

СХЕМА: МОДЕЛЬ ПРАВ ДОСТУПА LINUX

Кто владелец? К какой группе относится? Какие права у всех остальных?

> **Linux** использует модель **DAC (Discretionary Access Control)**
> Права назначаются владельцем файла.

1. ВЛАДЕЛЕЦ, ГРУППА И ОСТАЛЬНЫЕ



example.txt

Владелец (User)
создатель файла или тот, кто изменил владельца

Группа (Group)
пользователи, объединённые в группу

Остальные (Others)
все остальные пользователи системы

2. ТИПЫ ПРАВ

r (read)
чтение
просмотр содержимого файла или списка файлов в каталоге

w (write)
запись
изменение файла или создание/удаление файлов в каталоге

x (execute)
выполнение
запуск файла как программы или вход в каталог (cd)

Для каталога (директории)

- `r` — список файлов в каталоге (`ls`)
- `w` — создание/удаление файлов (`touch`, `rm`)
- `x` — вход в каталог (`cd`)

3. ЧИСЛОВОЕ И СИМВОЛЬНОЕ ПРЕДСТАВЛЕНИЕ

Символьное (пример)

```
-rw-r--r--
```

владелец | группа | остальные

Числовое (octal)

	w	x	Сумма
r	1	0	4
rw	1	0	6
r-x	1	0	5
rwx	1	1	7
---	0	0	0

Пример: `-rw-r--r--` = 4 2 4 = **644**

4. КТО ВЛАДЕЕТ И КАКАЯ ГРУППА

Команда	Описание
`ls -l example.txt`	показать владельца, группу и права

```
-rw-r--r--  1  alice  devteam  1024 May 20 10:30 example.txt
```

права | владелец | группа | размер | дата и время | имя

alice — владелец (user)
devteam — группа (group)

Пользователь принадлежит одной основной группе и может быть в дополнительных группах

5. КАК ПРОВЕРЯЕТСЯ ДОСТУП



6. СПЕЦИАЛЬНЫЕ ПРАВА

SetUID (4)
запуск с правами владельца файла
`rwsr-xr-x`

SetGID (2)
запуск с правами группы файла
`rwxr-sr-x`

Sticky bit (1)
удалять файлы в каталоге могут только владельцы файлов (и root)
`rwxrwxrwt`

Часто используется для общих каталогов, например `/tmp` (права 1777: `drwxrwxrwt`)

7. ВЛИЯНИЕ UMASK НА СОЗДАНИЕ ФАЙЛОВ

Максимальные права
файл 666 (`rw-rw-rw-`)
каталог 777 (`rwxrwxrwx`)

маска (umask)
0 2 2
−

Итоговые права
файл 644 (`rw-r--r--`)
каталог 755 (`rwxr-xr-x`)

Итог = максимальные права − umask

8. ВАЖНЫЕ КОМАНДЫ

`ls -l`	показать права, владельца, группу
`chmod`	изменить права доступа
`chown`	изменить владельца
`chgrp`	изменить группу
`umask`	показать/установить маску прав

```
$ chmod 755 script.sh
$ chmod u+x script.sh
$ chmod go-w file.txt
$ chmod 644 file.txt
```

`u` — владелец (user)
`g` — группа (group)
`o` — остальные (others)
`a` — все (all)

9. ЛУЧШИЕ ПРАКТИКИ

- Давайте минимум прав, необходимых для работы.
- Не используйте 777 без необходимости.
- Используйте группы для совместной работы.
- Регулярно проверяйте права важных файлов и каталогов.

ГЛАВНОЕ Права доступа в Linux определяются владельцем, группой и остальными.

Принцип: минимально необходимые права.

Команды: chmod, chown, chgrp, umask

Важно: root может обойти любые ограничения.

2. Права r, w, x для файлов

r для файла означает возможность читать содержимое.

w означает возможность изменять содержимое файла.

x означает возможность запускать файл как программу или скрипт.

Наличие **x** не делает текстовый файл правильной программой, но разрешает попытку запуска.

Пример прав на файл

Права **-rwxr-x---** означают: владелец может читать, писать и запускать.

Группа может читать и запускать, но не изменять.

Остальные не имеют доступа.

Такой режим подходит, например, для скрипта отдела.

3. Права r, w, x для каталогов

Для каталога r означает возможность видеть список имен.

w означает возможность создавать, удалять и переименовывать записи внутри каталога.

x означает возможность входить в каталог и обращаться к объектам по имени.

Права на каталог часто важнее прав на отдельный файл.

Почему x на каталоге критичен

Если у пользователя нет x на каталоге, он не сможет пройти через него к файлу.

Даже если файл внутри доступен, путь до него будет заблокирован.

Для доступа к /srv/project/report.txt нужен x на /srv и /srv/project.

Это частая ошибка при настройке общих папок.

4. Символьная запись прав

Символьная запись использует u, g, o, a и операции +, -, =.

u — владелец, g — группа, o — остальные, a — все.

Пример: chmod g+w report.txt добавляет группе право записи.

Пример: chmod o-r secret.txt забирает чтение у остальных.

Примеры символьного chmod

chmod u+x script.sh — разрешить владельцу запуск скрипта.
chmod g-w file.txt — убрать запись у группы.
chmod o= file.txt — убрать все права у остальных.
chmod a+r README.md — разрешить всем читать файл.

5. Числовая запись прав

Числовая запись складывает значения: r=4, w=2, x=1.

7 означает rwx, 6 означает rw-, 5 означает r-x, 0 означает ---.

Три цифры задают права владельца, группы и остальных.

Например, 750 означает rwx для владельца, r-x для группы, --- для остальных.

Как считать режим 640

- Первая цифра 6: владелец читает и пишет.
- Вторая цифра 4: группа только читает.
- Третья цифра 0: остальные не имеют доступа.
- Итог: -rw-r-----.
- Такой режим подходит для файла, который нельзя показывать всем пользователям.

6. Изменение прав chmod

chmod меняет режим доступа к файлу или каталогу.

Пример: **chmod 750 script.sh**.

Пример: **chmod g+w project.txt**.

После chmod обязательно выполнить ls -l и проверить итог.

Не используйте chmod 777 как универсальное решение: это почти всегда плохая практика.

chmod для каталогов

Для совместного каталога часто нужны права на вход и создание файлов.

Пример: chmod 2770 /srv/project.

Здесь 770 дает доступ владельцу и группе, а 2 включает SGID.

После изменения проверьте: ls -ld /srv/project.

Рекурсивный chmod

Ключ **-R** применяет права ко всем вложенным объектам.

Пример: **chmod -R g+rw /srv/project**

Это опасно, потому что права файлов и каталогов часто должны отличаться.

Перед -R сначала выполните find и посмотрите, что будет затронуто.

7. Изменение владельца и группы chown

chown меняет владельца и группу объекта.

Пример смены владельца: **sudo chown ivanov report.txt**

Пример смены владельца и группы: **sudo chown ivanov:developer report.txt**

Для смены только группы можно использовать **chgrp developer report.txt**

После изменения обязательно проверить ls -l.

Рекурсивный chown

chown -R меняет владельца у всего дерева каталогов.

Пример: **sudo chown -R root:developer /srv/project**

Команда полезна при подготовке общего ресурса.

Ошибка в пути может изменить владельцев большого количества файлов.

Перед выполнением проверьте путь через ls -ld и find.

ИЗМЕНЕНИЕ ВЛАДЕЛЬЦА И ГРУППЫ КОМАНДОЙ CHOWN

Команда chown позволяет изменить владельца (пользователя) и/или группу файла или каталога.

Для выполнения chown обычно требуются права root или права владельца файла с привилегиями sudo.

1. СИНТАКСИС КОМАНДЫ

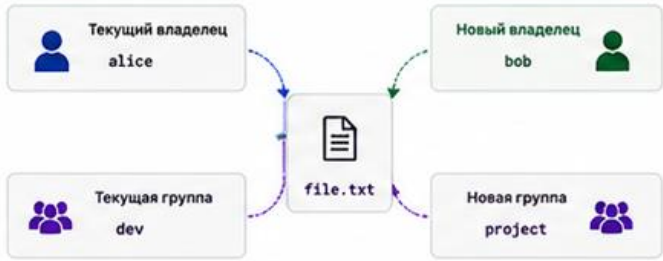
```
chown [опции] владелец[:группа] файл_или_каталог
```

- опции (необязательно)
- новый владелец (пользователь)
- новая группа (необязательно)
- файл или каталог, к которому применяется изменение

Примеры синтаксиса

```
chown user[illegible]          # изменить владельца на ot
chown user:group file.txt      # изменить владельца и группу
chown :group file.txt          # изменить только группу
chown user:  file.txt          # изменить только владельца (группа без изменений)
```

2. КАК РАБОТАЕТ CHOWN



chown изменяет информацию о владельце и/или группе (метаданные), но не затрагивает содержимое файла.

3. ОСНОВНЫЕ ВАРИАНТЫ И ОПЦИИ

Опция	Описание	Пример
(нет)	Изменить владельца и/или группу	`chown bob:dev file.txt`
`-R`	Рекурсивно для каталогов и их содержимого	`chown -R bob:dev /data`
`--from=старый`	Изменять только если текущий владелец соответствует указанному	`chown --from=alice bob file.txt`
`--reference=файл`	Скопировать владельца и группу из указанного файла	`chown --reference=/etc/passwd file.txt`
`-v`	Подробный вывод (показывает изменения)	`chown -v bob file.txt`
`--help`	Справка	`chown --help`
`--version`	Версия	`chown --version`

4. ПРИМЕРЫ ИСПОЛЬЗОВАНИЯ

Команда	Что делает
`chown bob file.txt`	Назначает пользователем bob
`chown bob:dev file.txt`	Назначает владельца bob и группу dev
`chown :dev file.txt`	Меняет только группу на dev
`chown alice: file.txt`	Меняет только владельца на alice
`chown -R bob:dev /project`	Рекурсивно меняет владельца и группу для /project
`chown --from=alice bob file.txt`	Меняет владельца на bob, только осли текущий alice
`chown --reference=/etc/passwd file.[illegible]`	Делает владельца и группу такими же, как у /etc/passwd

Будьте осторожны с -R!
Рекурсивное изменение может затронуть большое количество файлов.

5. ПРИМЕР РАБОТЫ

Исходные данные:

```
$ ls -l file.txt
-rw-r--r-- 1 alice dev 1024 May 20 10:00 file.txt
```

Изменим владельца на bob:

```
$ sudo chown bob file.txt
$ ls -l file.txt
-rw-r--r-- 1 bob dev 1024 May 20 10:00 file.txt
```

Изменим владельца и группу:

```
$ sudo chown bob:project file.txt
$ ls -l file.txt
-rw-r--r-- 1 bob project 1024 May 20 10:00 file.txt
```

Рекурсивное изменение:

```
$ sudo chown -R bob:dev /data
$ ls -ld /data
drwxr-xr-x 5 bob dev  4096 May 20 11:00 /data
```

Копирование владельца и группы из другого файла:

```
$ sudo chown --reference=/etc/passwd file.txt
$ ls -l file.txt
-rw-r--r-- 1 root root 1024 May 20 10:00 file.txt
```

Для изменения владельца или группы требуется:
- быть root (через sudo)
- или иметь специальные привилегии (CAP_CHOWN)

6. КТО МОЖЕТ ИСПОЛЬЗОВАТЬ CHOWN?

1. Root (администратор)
- Может менять владельца и группу любых файлов
- Использует: `sudo chown ...`

2. Обычный пользователь
- Может менять владельца на себя
- Может менять группу на одну из своих групп
- Имеет право записи на файл/каталог

Обычный пользователь НЕ может назначить владельцем другого пользователя, если у него нет соответствующих прав.

7. ПОЛЕЗНЫЕ КОМАНДЫ РЯДОМ

`ls -l`	показать владельца и группу
`stat файл`	подробняя информация о файле
`id`	показать текущего пользователя и группы
`groups`	показать группы текущего пользователя
`getfacl файл`	показать ACL (дополнительные права)

8. ВАЖНЫЕ ЗАМЕТКИ

- chown изменяет только владельца/группу, но не права доступа.
- Используйте -R с осторожностью, особенно в системных каталогах.
- После изменения владельца могут измениться права доступа к файлу.
- Проверяйте результат командой `ls -l` или `stat`.
- Для массового изменения лучше использовать скрипты и `--from`.

9. ПРИМЕРЫ ИЗ ПРАКТИКИ

Задача	Решение
Передать файл другому пользователю	`sudo chown bob file.txt`
Передать каталог с проектом группе dev	`sudo chown -R :dev /home/alice/project`
Назначить текущего пользователя владельцем файла	`sudo chown $USER file.txt`
Копировать владельца и группу из эталонного файла	`sudo chown --reference=/etc/passwd file.txt`

КРАТКО

chown меняет владельца и/или группу → Используйте формат владелец[:группа] → -R для каталогов и вложенных файлов → Требуются права root или соответствующие привилегии → Проверяйте результат командой `ls -l`

8. SUID

SUID — специальный бит, при котором программа запускается с правами владельца файла.

Классический пример: passwd, которому нужно менять защищенный файл shadow.

В ls -l SUID отображается как s в правах владельца.

Ставить SUID на собственные скрипты опасно и обычно не нужно.

SGID

SGID на файле запускает программу с группой файла.

SGID на каталоге заставляет новые файлы наследовать группу каталога.

Это полезно для совместных каталогов группы.

Пример: **chmod g+s /srv/project** или **chmod 2770 /srv/project**

Sticky bit

Sticky bit на каталоге разрешает удалять файл только владельцу файла, владельцу каталога или root.

Классический пример: /tmp.

В ls -ld sticky bit отображается как t в правах остальных.

Пример: **chmod +t /srv/exchange**

Это полезно для каталогов обмена, где многие могут создавать файлы.

СПЕЦИАЛЬНЫЕ БИТЫ: SUID, SGID, STICKY BIT

Расширяют стандартные права Linux для обеспечения безопасности и совместной работы.

Специальные биты устанавливаются на исполняемые файлы или каталоги и изменяют поведение прав доступа.

1. ЧТО ТАКОЕ СПЕЦИАЛЬНЫЕ БИТЫ?

Они дополняют стандартные права (rwx) и задаются в старшем бите поля прав для владельца, группы или остальных.

Бит	Обозначение	Тип объекта	Эффект
SUID	u+s (4xxx)	файлы	Процесс выполняется с правами владельца файла
SGID	g+s (2xxx)	файлы/каталоги	Файлы наследуют группу каталога / Процесс выполняется с правами группы файла
Sticky bit	o+t (1xxx)	каталоги	Удалять и переименовывать файлы в каталоге может только владелец файла, владелец каталога или root

2. КАК ОТОБРАЖАЮТСЯ ПРАВА (ls -l)

Специальные биты заменяют соответствующие биты исполнения (x) и отображаются в виде букв.

Бит установлен	Отображение в правах	Пример (права)
SUID	s вместо x в поле владельца	rwsr-xr-x
SGID (файл)	s вместо x в поле группы	rwxr-sr-x
SGID (каталог)	s вместо x в поле группы	rwxr-sr-x
Sticky bit	t вместо x в поле остальных	rwxr-xr-t

Если у бита нет права на исполнение (x), отображается заглавная буква:
SUID → S, SGID → S, Sticky → T

3. УСТАНОВКА И ПРОСМОТР БИТОВ

Установка (chmod)

```
chmod u+s файл          # SUID
chmod g+s файл          # SGID (файл)
chmod g+s каталог       # SGID (каталог)
chmod +t каталог        # Sticky bit
```

Числовой вид (chmod)

```
chmod 4xxx файл         # SUID
chmod 2xxx файл/каталог # SGID
chmod 1xxx каталог      # Sticky bit
```

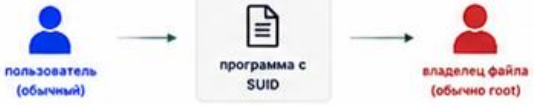
Просмотр прав

```
ls -l файл_или_каталог
-rwsr-xr-x 1 root root 72000 Jan  1 10:00 /usr/bin/passwd
drwxr-sr-x 2 dev  dev  4096 May 20 10:00 /srv/project
drwxrwxrwt 10 root root 4096 May 20 10:00 /tmp
```

Расшифровка:
s в первом поле → SUID
s во втором поле → SGID
t в третьем поле → Sticky bit

4. SUID (SET USER ID)

Процесс, запущенный пользователем, выполняется с правами владельца файла.



Пример: /usr/bin/passwd

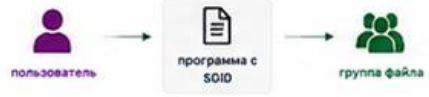
```
$ ls -l /usr/bin/passwd
-rwsr-xr-x 1 root root 72000 Jan  1 10:00 /usr/bin/passwd
$ ./passwd
# программа выполняется с правами root,
# но пользователь не становится root
```

Используется для административных задач без входа в систему под root.
Опасность: уязвимые программы с SUID могут быть использованы для повышения привилегий.

5. SGID (SET GROUP ID)

Для файлов: процесс выполняется с правами группы файла.
Для каталогов: новые файлы наследуют группу каталога.

SGID на файле



Пример:

```
$ ls -l /usr/bin/wall
-rwxr-sr-x 1 root tty 5480 Jan  1 10:00 /usr/bin/wall
$ ./wall "Привет, мир!"
# программа выполняется с группой tty
```

SGID на каталоге



Пример:

```
$ ls -ld /srv/project
drwxr-sr-x 2 dev dev 4096 May 20 10:00 /srv/project
$ touch /srv/project/file1.txt
$ ls -l /srv/project/file1.txt
-rw-r--r-- 1 user  dev   0 May 20 10:05 file1.txt
# группа файла = dev (наследована от каталога)
```

Полезно для совместной работы в проектах/каталогах.

6. STICKY BIT

На каталоге: только владелец файла, владелец каталога или root могут удалять или переименовывать файлы в этом каталоге.




```
пример:
$ ls -ld /tmp
drwxrwxrwt  10 root root  4096 May 20 10:00  /tmp
$ rm /tmp/file_другого_пользователя
rm: Operation not permitted
```

Классический пример: /tmp. Пользователи могут создавать файлы, но не могут удалять чужие.

7. ТАБЛИЦА БЫСТРОГО ОБЗОРА

Бит	Применяется к	Устанавливается	Отображается как	Назначение
SUID	файлы	chmod u+s файл (или 4xxx)	s в поле владельца	Выполнение с правами владельца файла
SGID	файлы/каталоги	chmod g+s объект (или 2xxx)	s в поле группы	Выполнение с группой файла / наследование группы
Sticky bit	каталоги	chmod +t каталог (или 1xxx)	t в поле остальных	Защита файлов от удаления другими пользователями

8. ПРОВЕРКА И СНЯТИЕ БИТОВ

Проверка

```
$ ls -l файл_или_каталог
```

Снятие

```
chmod u-s файл          # снять SUID
chmod g-s файл/каталог  # снять SGID
chmod -t каталог        # снять Sticky bit
```

ЛУЧШИЕ ПРАКТИКИ

- Устанавливайте SUID только на проверенные системные программы.
- Не устанавливайте SUID/SGID на свои скрипты без необходимости.
- Регулярно проверяйте файлы с SUID/SGID: `find / -perm -4000 -type f 2>/dev/null`
- Используйте Sticky bit для общих каталогов (например, /tmp, /var/tmp).

ВАЖНО

Неправильное использование специальных битов может привести к повышению привилегий и угрозам безопасности системы.

9. Совместный каталог группы

Для совместной работы создают группу, каталог и назначают группу владельцем каталога.

Пример: sudo groupadd developer.

Пример: sudo mkdir -p /srv/project.

Пример: sudo chown root:developer /srv/project.

Пример: sudo chmod 2770 /srv/project.

Проверка совместного каталога

Проверьте каталог: ls -ld /srv/project.

Ожидаемый режим похож на drwxrws--- root developer.

Добавьте пользователя в группу: sudo usermod -aG developer ivanov.

Пользователь должен перелогиниться, чтобы получить новую группу.

После входа создайте тестовый файл и проверьте его группу.

Типовые ошибки прав

- Нет x на каталоге, поэтому пользователь не может пройти к файлу.
- Забыли -a в usermod -G и удалили старые дополнительные группы.
- Поставили chmod 777 вместо настройки группы.
- Не включили SGID, и новые файлы получают не ту группу.
- Проверяли доступ под root, поэтому не заметили ошибку для обычного пользователя.

Почему права доступа — основа безопасности Linux

- Права определяют, кто может читать, изменять и запускать файлы.
- Неверные права могут открыть конфиденциальные данные или сломать работу службы.
- Службы часто работают от отдельных системных пользователей.
- Администратор должен настраивать права минимально необходимыми, а не максимально удобными.

Как читать строку прав в ls -l

- Первый символ показывает тип объекта: файл, каталог или ссылка.
- Далее идут три группы прав: владелец, группа, остальные.
- r означает чтение, w — запись, x — выполнение или вход в каталог.
- Владелец и группа показывают, кому эти права принадлежат.

Права каталога отличаются от прав файла

Для файла x означает возможность запуска как программы или скрипта.

Для каталога x означает возможность войти в каталог и обращаться к объектам внутри.

r для каталога позволяет видеть список имен.

w для каталога позволяет создавать и удалять записи, если есть x.

Итоги лекции

- Команды Linux нужно понимать через объект, с которым они работают.
- Любое изменение системы завершается проверкой результата.
- Опасные команды требуют предварительной проверки пути, прав и текущего пользователя.
- Отчет по лабораторной должен содержать не только команды, но и выводы по результатам.

Контрольные вопросы

1. Как читать строку ls -l?
2. Чем права r, w, x для каталога отличаются от прав для файла?
3. Как посчитать числовой режим 750?
4. Для чего SGID используется на совместном каталоге?
5. Почему chmod 777 почти всегда плохое решение?